Applied AI in Python

UV
Wintersemester 2026/27

University of Salzburg

Dominik Geng

Monday, 12:00–15:00 Rechnerraum (JAK2EG0.09)

1 Introduction

Welcome to the Applied AI in Python Course 2026/27. These lecture notes are designed to serve as the theory backbone of the course and will be updated as the course goes along.

We will start with a short introduction to AI to get an understanding of how it historically evolved. Over the first couple of lectures we will then mainly deal with the fundamentals of `Python`, what distinguishes it from other programming languages and its basic data structures. Then, we will deal with actual data and introduce `NumPy`, `Python`'s workhorse for numerical computations. We'll meet machine learning in terms of regression and classification, mainly in a *supervised* manner. We then introduce and build *Backpropagation* from scratch, introduce `PyTorch` and create our first *neural networks*. We close with *convolutional* neural networks, a neural network specialized for locally correlated data (e.g., signals, images).

Throughout the course, you will work towards a project on MNIST-1D (Greydanus and Kobak, 2024), a small, one-dimensional variant of the classic MNIST digits. It is small enough to run experiments on a laptop within minutes, yet it clearly separates models by how well their built-in assumptions fit the data. The project guidelines, with all deadlines and requirements, are provided separately in the course repository and on Blackboard.

2 A short primer on terms and history

2.1 What is Artificial Intelligence?

What is Artificial Intelligence? A reasonable question but also an article written by AI pioneer John McCarthy (2007). There, he gives the answer

> *It is the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but AI does not have to confine itself to methods that are biologically observable.*

One could go on and wonder what then even intelligence is, if it is well-defined and any measures used for us humans even make sense for computers (cf. McCarthy, 2007). As McCarthy writes in his article, such definitions are not consensus among the AI research community. Chollet and Watson (2025), for instance, define AI more concisely as

> *the effort to automate intellectual tasks normally performed by humans*

which, in contrast to McCarthy, does not mention the reverse direction, i.e., studying with such means the human intelligence. Given machines were reasonable in solving problems in tradition reserved for humans, this raises a question.

2.2 Can machines think?

Alan Turing asks this question as he starts off his seminal paper on "Computing Machinery and Intelligence" (Turing, 1950).

Very quickly, however, he abandons it on grounds of the impossibility of defining its constituents well enough: What is a "machine"? And, arguably more difficult, what is "thinking"? He replaces it instead with the *Imitation Game*, which we now commonly refer to as the *Turing Test*.

In its first formulation, the imitation game hosts 3 players: a man A and a woman B, hidden from an interrogator C. The goal of the interrogator is to find out who among A and B is the woman. While A wants to deceive C into thinking that he is, B wants the truth to be known, and convince C that she is. Naturally, such a game would have to be played via channels of communication, e.g. typewritten, that do not readily give away the player's identity other than through the messages' content.

The article goes on to propose another incarnation of the game, one where a machine replaces A and the interrogator's goal becomes to discern who of A and B is, effectively, a human. How often, especially in comparison with the first formulation of the game, would the interrogator be misled by the machine?

McCarthy (2007) argues that the Turing Test should be regarded as a one-sided test: A machine passing it should be considered intelligent. One that does not, however, should *not* automatically be labelled not intelligent. It may have other branches of intelligence but solely lack the ability to emulate humans.

Turing discusses a number of possible objections to whether this test could truly attest to a machine's intelligence. Apart from the objections, already then, he discusses the possibility for machines to learn: reward and punishment signals would not suffice, so additional "unemotional" communication channels should carry instructions in a symbolic language, which greatly reduces the number of rewards and punishments.

The Turing Test should *not* be thought of as a rigorous measure to assess a machine's intelligence. It is nonetheless a philosophical treatment and brings to attention the difficulty of defining (artificial) intelligence. Turing ends his article with a most generic, yet ever suitable,

> *We can only see a short distance ahead, but we can see plenty there that needs to be done.*

2.3 The birthplace of AI research

Enter 1955 when John McCarthy, Marvin Minsky, Nathaniel Rochester and Claude Shannon put forward a proposal for the Dartmouth Summer Research Project on Artificial Intelligence (McCarthy et al., 1955).

In the proposal, the authors propose ideas that are still of high relevance today. Among those being the learning procedure in a trial and error way, the necessity of a random, but not completely random, element, the study of neural networks or the forming of internal abstraction and concepts inside the algorithm.

The summer research project did indeed take place in the following summer of 1956 and included high-profile attendees, many of whom became known as pioneers of the field. Even though, and understandably so, the event did not solve all the problems it set out to, it kickstarted the field of AI research and is therefore nowadays often referred to as its birthplace.[1]

Since then, there have been highs and lows in the field, with the lows often referred to as AI winters. The first one (roughly 1974–1980) had several causes: disappointed expectations, e.g. in machine translation; a shift of US military funding towards applied research; and the critical Lighthill report (1973), after which the UK cut AI funding at all but two universities. For neural networks in particular, Minsky, himself a founding father of AI, is often held responsible: in their book "Perceptrons" (Minsky and Papert, 1969), he and Papert proved rigorously what single-layer perceptrons can represent (think "linearly separable") and what they cannot (think "XOR"), and judged multilayer extensions to be likely "sterile". Funding for neural networks dried up until backpropagation became widely known in the 1980s, although Minsky and Papert themselves argued that progress had already stalled.

The revival of neural networks began in 1986, when Rumelhart, Hinton and Williams demonstrated how networks with hidden layers can be trained by *backpropagation*: the error of the output is propagated backwards through the network by the chain rule, which tells every weight how to change (Rumelhart et al., 1986). The idea itself was older. Its mathematical core, the reverse mode of automatic differentiation, goes back to Linnainmaa (1976), and Werbos had already applied it to neural networks in his 1974 dissertation (Werbos, 1974). We will build backpropagation ourselves later in this course.

The next breakthrough was driven by data and hardware. In 2012, *AlexNet*, a deep convolutional network trained for five to six days on two graphics processing units (GPUs), won the ImageNet image recognition challenge with a top-5 error rate of 15.3 %, far ahead of the second-best entry with 26.2 % (Krizhevsky et al., 2012). Since then, deep learning has dominated computer vision, and GPUs have become the workhorses of machine learning. Finally, in 2017, the *transformer*, an architecture "based solely on attention mechanisms, dispensing with recurrence and convolutions entirely" (Vaswani et al., 2017), laid the foundation of today's large language models.

2.4 What is Machine Learning

Having stated that Artificial Intelligence is a very broad endeavor, its most known domain today probably is *Machine Learning*. While both Turing and the Dartmouth conference discussed "learning machines", much of early AI research focused on systems equipped with large corpora of handwritten rules. These lay out how to act (e.g. a chess move) in face of certain data (e.g. a preceding opponents chess move, the state of the board, etc.). This methodology, often running under the name *symbolic AI*, is based on a classical principle of programming: Rules applied to given data yield answers.

Machine learning can perhaps best be understood as a different paradigm: Data

[1] To be fair to history, a number of researchers started to work on AI after the second world war, Turing probably being the first and most prominent as he gave a lecture on it already in 1947.

together with answers lets the program *infer* the rules. Or, as Chollet and Watson (2025) put it,

> *... machine learning is, concisely: searching for useful representations and rules over some input data, within a predefined space of possibilities, using guidance from a feedback signal.*

The deep in "deep learning", a subfield of machine learning, then merely refers to building up of several successive layers of *hidden* representations.

Before we explore deep learning and its intricacies, however, we introduce `Python`, as of writing, one of, if not the, most popular programming languages in general and specifically for artificial intelligence applications.

3 Python

`Python` is often referred to as a scripting language to distinguish it from other languages. This raises the question, what it even means to be a scripting language. Different people have different conceptions of a scripting language, e.g., (1) it being relatively simple and easy to read, (2) the language serving mostly as an interface to the operating systems, say, as a file management and command line utility and thereby being a more versatile bash scripting language, or (3) a language mainly implemented on a high level in order to orchestrate smaller lower level pieces. `Python` surely can achieve all of the latter but, in fact, is much more. It supports a procedural, functional and object-oriented approach to programming, can easily be used to build large software projects on its own, with an extended application field toward Web Development, Databases, Numeric and Scientific programming and, notably, AI.

On the other hand, it is often called an "interpreted" language. This, however, can also not be said in generality. Its default implementation, `CPython`, is indeed interpreted, does however support (experimental) just-in-time (JIT) compilation in more recent versions. Nevertheless, other implementations exist such as `PyPy`, an interpreter written in a restricted subset of `Python` (RPython) that adds a JIT compiler, or `Jython` that is based on `Java` as opposed to `C`. Furthermore, `Numba` runs inside `CPython` and is a library that JIT-compiles selected numerical computations. `Cython` is a superset of `Python` which reads like a hybrid between `Python` and `C`, is compiled ahead of time to `C` extension modules which then run in `CPython`.

Calling `Python` a scripting or an interpreted programming language is therefore not useful to distinguish it from other languages. It is more helpful to describe `Python` as a *dynamically typed* in contrast to the *statically typed* and usually pre-compiled languages. This is both the blessing and the curse of `Python`: it makes the language comparatively easy to read, understand and implement while, at the same time, introducing a large computational overhead at lower levels, as type and object consistency has to be checked again and again.

Lutz (2025) summarizes `Python` as

> *A general-purpose programming language that blends procedural, functional, and object-oriented paradigms and accelerates software development by reducing complexity.*

Likewise, the Python Software Foundation calls it an easy to learn, powerful programming language, sometimes considered very high-level, with a simple but effective approach to object oriented programming (OOP) (Python Software Foundation, 2026b).

This leads us to the largest upshot of `Python`: its simplicity and ability to reduce complexity.

Usually, a program can be written in `Python` using far fewer lines than in other languages while being more readable. This does not optimize machine execution speed but it aims to minimize development time. A further advantage is that, besides its extensive standard library, it is easily extended to other domains such as numerical and scientific programming (`NumPy`, `SciPy`) Web development (e.g., via Django), Databases (e.g., via various SQL(ite) integrations) and recently foremost Machine Learning (e.g., via `PyTorch`, `Keras`, `TensorFlow`). Many such third party libraries run performance optimized `C` code under the hood such that the performance drawback need not be factored in (as long as execution remains in the library; a `Python for`-loop over `NumPy` computations does inherit `Python`'s slowness).

Moreover, `Python` as an executable and source code freely distributed online, works on the major operating systems Windows, MacOS, Linux but also on mobile devices, high performance clusters, microcontrollers and more. It is also portable in terms of its functionality. For instance, its file system handling can be written on any of the major OSs and be executed on any other.

3.1 Python as an interpreted language

In its most used implementation (`CPython`), `Python` is *not* compiled *ahead of time* (AOT) but *interpreted*. For starters, compare `Python` to `C`. `C` is naturally an AOT-compiled language, meaning that it is compiled to machine code in a process that is run before and separate from execution of the code. Consider a "Hello world" program written in `C`:

```c
#include <stdio.h>

int main(void) {
    printf("Hello world");
}
```

The same functionality expressed in a `Python` script

```python
print("Hello world")
```

looks simpler: No imports, semicolons, curly braces, function definitions and even a letter less on the print function.

One often hears that `Python` is an *interpreted* language and this interpretation would be done line by line. We can check whether it is indeed so straightforward as going from `Python` to machine code in a line by line manner.

Consider the following Python code in a file linebyline.py

```
a = 2 / 0
```

This code contains a ZeroDivisionError, which is a runtime error, i.e., an error that is thrown when the code is actually executed by the machine.

To confirm that the ZeroDivisionError indeed is a runtime error and not thrown before execution, we can prepend the code by a valid print instruction,

```
1 print("Looks good to me")
2 a = 2 / 0
```

Running this code will output

```
> python3 linebyline.py
Looks good to me
Traceback (most recent call last):
  File ".../line_by_line/linebyline.py", line 2, in <module>
    a = 2 / 0
        ~~~~~
ZeroDivisionError: division by zero
```

The script is actually executed, otherwise we would not see the "Looks good to me" print. Let's add a bit more to the script:

```
1 print("Looks good to me")
2 a = 2 / 0
3
4 print("How about now?)
```

The ending quotation marks are missing but the interpreter should not care, as it runs into the ZeroDivisionError at runtime first. Executing the script, we get however

```
> python3 linebyline.py
  File ".../line_by_line/linebyline.py", line 4
    print("How about now?)
          ^
SyntaxError: unterminated string literal (detected at line 4)
```

The interpreter actually catches the SyntaxError *before* the code is executed which, in turn, means that running Python code cannot be so simple as executing line after line.

It is a bit more involved. The Python code (.py) is first translated into an abstract syntax tree. This abstract syntax tree is then compiled into Python byte code (.pyc). This bytecode sits somewhere in between source and machine code, grouping source code statements to (still higher level) *operation codes* (opcodes). A *Python Virtual Machine* (PVM) is then the final part of the python interpreter, branching on the opcode and executing it (the machine code of the individual opcodes was compiled when CPython

was built). For `CPython`, this PVM is just a large `C` program that iterates through the bytecode instructions. Each opcode dispatches to `C` code. In the process from `.py` file to `Python` byte code, some checks can cheaply be made, as e.g. those for proper syntax. Others, as the `ZeroDivisionError`, will only be noted at actual time of execution.

In a C file, `linebyline.c`,

```c
#include <stdio.h>

int main(void) {
    int c = 2 / 0;
    printf("%i\n", c);
    return 0;
}
```

this would not have gone unnoticed:

```
> cc linebyline.c -o linebyline.out
linebyline.c:4:15: warning: division by zero is undefined [-Wdivision-by-zero]
    4 |     int c = 2 / 0;
      |               ^ ~
1 warning generated.
```

Nevertheless, the binary machine code is compiled. Indeed, as the compiler warns us, the result is ill-defined[2]

```
> ./linebyline.out
1531547984
```

So now we've seen that there is more to interpreting than simply converting `Python` to machine code line by line. But, this magical interpreter, in what language is it itself written?

It depends. As we've just seen, the default implementation is `CPython` which more or less means that a big C file does the interpreting and executing. Other implementations, such as `PyPy` or `Jython`, and JIT-compiling libraries like `Numba` were already mentioned above. Note that from version 3.13 onward, `CPython` also comes with its own, experimental JIT compiler, which, however, is not enabled by default.

When `Python` is in the end interpreted by, say, `C` which is arguably a very fast language, why is `Python` slow nevertheless? The answer lies to a large degree in its dynamic nature, its "everything goes" approach. E.g. declaring a variable, `a=2` or `b="some string"` or

[2] In `C`, division by zero is *undefined behavior*: the language standard does not prescribe any result, so the compiler may do anything. `2 / 0` only becomes a compile *error* where `C` requires a compile-time constant, e.g. `static int c = 2 / 0;`; inside a function, as here, compilers merely warn. What the program then does depends on compiler and processor: here, `clang` skips the division altogether and returns whatever value happens to sit in a register (hence the arbitrary number below); `gcc` performs the division, which yields 0 on ARM processors (e.g. Apple Silicon) and crashes the program on x86 processors (Intel, AMD).

`c=2.0`, will go easy on us as we do not need to worry about typing and declarations. The price has to be paid in the end and that amounts to a myriad of type checks and reference counts.

In `Python`, a *variable* (also called name) merely *references* an *object*. We can think of the variable as a space in memory that points to the object, similar to a pointer in `C` that is automatically dereferenced. The object is space in memory with the value itself and two header fields, one for the type and one for the reference count. The type header is actually a reference to the *type* (say `type int`) in memory. The reference count counts the number of references to the object and is used to automatically free up space in case an object is not referenced anymore.

Indeed, every value in `Python` is a full blown `PyObject` at the `C` level (speaking for the `CPython` implementation here) and whenever two `PyObject`s meet, e.g. in an addition `a+b`, their types have to be checked.

To get an idea of all the operations that go on, one can *disassemble* `Python` programs to peek under the hood by using the `dis` module from the standard library. Consider for instance a function that reads the attribute `x` of its first argument and the first element of its second argument, and adds the two:

```
1 import dis
2
3 def add(a, b):
4     return a.x + b[0]
5
6 dis.dis(add, show_caches=True)
7
```

Running this script with `CPython` 3.13, the version uv sets up for this course, returns[3]

```
> python3 disassemble.py
  3           RESUME                   0

  4           LOAD_FAST                0 (a)
              LOAD_ATTR                0 (x)
              CACHE                    0 (counter: 0)
              CACHE                    0 (version: 0)
              CACHE                    0
              CACHE                    0 (keys_version: 0)
              CACHE                    0
              CACHE                    0 (descr: 0)
              CACHE                    0
              CACHE                    0
              CACHE                    0
```

[3]Bytecode is an implementation detail of `CPython`: instructions may be added, removed or changed between versions (Python Software Foundation, 2026a). With `Python` 3.14, for instance, which runs on the university's lab PCs, the same script already yields different opcodes.

```
              LOAD_FAST                1 (b)
              LOAD_CONST               1 (0)
              BINARY_SUBSCR
              CACHE                    0 (counter: 0)
              BINARY_OP                0 (+)
              CACHE                    0 (counter: 0)
              RETURN_VALUE
```

Each all-capital letter term on the left hand side is an opcode, the symbolic name for a single instruction. Each opcode corresponds to *one* small behavior in the interpreter ("load a variable", "add two objects", "return from a function", etc.). Each such opcode is further identified with an integer (which should be visible in the `.pyc` byte code). All these opcodes are then handled inside a single large `C` function in the `CPython` interpreter with one big *dispatch* loop that branches on the opcode value and runs the appropriate block of C code inline.

The `Python` virtual machine is a *stack machine*: every instruction takes its operands from a stack of values and pushes its result back onto it. Read from top to bottom, the listing thus says: push `a` and replace it by `a.x`; push `b` and the constant `0` and replace both by `b[0]`; replace the two remaining values by their sum and return it. (`RESUME` is an internal no-op that performs checks for tracing, debugging and optimization.)

The `CACHE` entries are not instructions of their own but reserved space directly in the bytecode. Since version 3.11, `CPython` uses them for a *specializing adaptive interpreter* (Shannon, 2021): once a function has been executed (in fact, already from its second call on), generic instructions are replaced by variants specialized to the types actually observed, and the data these variants need is stored in the caches. After calling `add` with an object whose attribute `x` is an integer and a list of integers, `dis.dis(add, adaptive=True)` shows `LOAD_ATTR_INSTANCE_VALUE`, `BINARY_SUBSCR_LIST_INT` and `BINARY_OP_ADD_INT` in place of the generic opcodes. `BINARY_OP_ADD_INT`, for instance, merely checks that both operands are indeed integers and then adds them directly, instead of first working out what `+` means for the two objects at hand. If the types change, say to floats, the interpreter falls back to the generic instruction and eventually specializes anew, here to `BINARY_OP_ADD_FLOAT`. The type checks described above thus remain, but they become cheap as long as the code is *type-stable*, which most code in practice is.

One should clarify, that the saving of byte-code is purely for optimizing *import* time. If a sole `Python` script is run, that does not import any other `Python` modules, the `pyc` byte code will be kept in memory and not written to disk. On the other hand, if a larger program, importing other files, is interpreted, bytecodes *of the imported* files will be saved and be reused upon later executions (unless source code or interpreting python version have changed). These can be inspected in a `__pycache__` directory, which is saved next to the imported module's source file.

3.2 Types

Everything in `Python` is an object and an object boils down to a piece of memory where the actual value is saved with additional headers (the refcount and type as mentioned

earlier). Its type specifies which operations it can perform, and, how these operations are performed. For instance, addition `+` is implemented for all sorts of numbers but also, e.g., lists (`[1, 2] + [3, 4] = [1, 2, 3, 4]`) and strings (`"filename" + ".pdf" = "filename.pdf"`).

Names carry no type themselves; the type belongs to the object a name refers to and is determined when the object is created, echoing the dynamic typing paradigm. A name can later be rebound to an object of a different type. At the same time, however, `Python` is *strongly typed*. That is, types can only perform the actions defined for them and we cannot treat different types in the same manner. For instance, you cannot add a string and a list object.

3.2.1 Strings

Strings in `Python` are our first encounter of a *Sequence*, i.e., a *positionally ordered* collection of objects. Strings serve as the data type to work with text and, as sequences, serve as an implementation of ordered characters (`char` in some programming languages). We indicate the string type `str` by using double or single quotes.

```
>>> s = 'name'
```

As sequences, they support the extraction of individual components (characters for strings), where the index in square brackets indicates the positional offset to the start of the sequence. In other words, `Python` is 0-indexed and we start to count with 0:

```
>>> s[0]
'n'
```

Strings also support *slicing* in order to extract whole chunks out of the object.

```
>>> s[0:3]
'nam'
```

As you see in the example, the syntax `s[i:j]` translates to, "yield everything in `s` from `i` to `j` excluding `j`". By default and if not specified the beginning of a slice is 0 and the end is the length of the sequence, i.e., `len(s)`, so that the final character is included.

```
>>> s[:3]
'nam'
>>> s[0:]
'name'
>>> s[:]
'name'
```

It generally holds for `Python` sequences that indexing requires to be in bounds while slicing clips. That is, a single index has to be smaller than the object's length, while a slice going out of bounds will be clipped to the available size and may thus be empty (e.g., `s[10:]` is `''`).

Importantly, strings in `Python` are *immutable*. One cannot change their components but only make new objects. Every slice we just returned is, in fact, a new object.

```
>>> s[0] = 't'  # raises a TypeError as Python strings are immutable
```

3.2.2 Lists

Lists are probably the Sequence that is easiest to understand and to work with in `Python`. In contrast to usual *array* implementations in other programming languages, neither the type of their elements nor their size is fixed. This means, (1) that one list can hold many different types of elements (e.g., `int`s and `str`s) and (2) that lists are *mutable*, i.e., they can grow, shrink and be changed in place. Furthermore, they can be nested, i.e., hold lists of lists etc. Lists are most naturally initialized with square brackets.

```
>>> l1 = [0, 1, 2]
>>> l1[0] = 5  # lists, unlike strings, can be modified in place
>>> l1
[5, 1, 2]
```

A quick way to initialize lists almost reading like natural language is *list comprehension*.

```
>>> l2 = [i for i in range(10)]  # creates a list with integers from 0 to 9
```

3.2.3 Dictionaries

`Python` dictionaries `dict`s are its most prominent representative of a *mapping*. A `dict` is mostly initialized with braces and uses *keys* to index *values*.

```
>>> d = {"a":1, "b":2, "c":3}
>>> d["a"]
1
```

While new key-value pairs can be defined by indexing with a new key, accessing an undefined key raises a `KeyError`. `Python` `dict`s are also mutable and can be nested (dictionaries holding dictionaries). Keys must be hashable (see the section on sets below) and `dict`s remember the insertion order of their keys. Similar to list comprehension, there is *dict comprehension*.

```
>>> d = {k: v for (k, v) in zip(["a", "b", "c"], [1, 2, 3])}
```

3.2.4 Tuples

Tuples `tuple` are sequences just like lists, but they are immutable. They support the usual sequence methods, like indexing, counting and slicing. They can also be nested and contain elements of various datatypes.

```
>>> t1 = ((1, 2),  (3, 4, 5), (6, 7, 8, 9))
>>> t2 = (1, 2, "a", [1, 2, 3])
```

Their immutability is their defining property: it makes them suitable as fixed records and as hashable dictionary keys.

```
>>> t = (1, 2, 3)
>>> t[0] = 4  # will raise an error
```

Importantly, they are not distinguished by parentheses but by commas.

```
>>> not_a_tup = (1)
>>> a_tup = 1,
```

3.2.5 Sets

Sets are neither sequences nor mappings (but can be iterated over).

```
>>> s = {1, 2, 3}
```

Sets themselves are mutable, but since they are unordered they cannot be indexed.

```
>>> s.add(4)
>>> s
{1, 2, 3, 4}
```

Sets importantly do not hold duplicate values.

```
>>> l = [1, 1, 1, 2, 3, 2, 2, 3, 4]
>>> s1 = set(l)
>>> s1
{1, 2, 3, 4}
```

Elements are usually immutable, but strictly speaking they must be *hashable*. The hash value of an element must be conserved over its lifetime, otherwise, it could not be found again. In essence, sets are hash tables, which is why element lookup is fast (on average $\mathcal{O}(1)$).

References

François Chollet and Matthew Watson. *Deep Learning with Python*. Manning, third edition, 2025. ISBN 9781633436589. URL https://deeplearningwithpython.io/.

Sam Greydanus and Dmitry Kobak. Scaling down deep learning with MNIST-1D. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://github.com/greydanus/mnist1d.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In F. Pereira, C. J. Burges, L. Bottou, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012. URL https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.

Seppo Linnainmaa. Taylor expansion of the accumulated rounding error. *BIT*, 16(2): 146–160, 1976. doi: 10.1007/BF01931367.

Mark Lutz. *Learning Python*. O'Reilly Media, Sebastopol, CA, sixth edition, February 2025. ISBN 9781098171308. URL https://learning-python.com/about-lp6e.html.

John McCarthy. What is Artificial Intelligence?, December 2007. URL https://www-formal.stanford.edu/jmc/whatisai.pdf. https://www-formal.stanford.edu/jmc/whatisai.pdf, accessed on 2025-06-10.

John McCarthy, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon. A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence, August 1955.

Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.

Python Software Foundation. dis — disassembler for Python bytecode. https://docs.python.org/3.13/library/dis.html, 2026a. Python 3.13 documentation, accessed on 2026-09-29.

Python Software Foundation. The Python Tutorial, 2026b. URL https://docs.python.org/3/tutorial/index.html.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.

Mark Shannon. PEP 659 – specializing adaptive interpreter. https://peps.python.org/pep-0659/, 2021. accessed on 2026-09-29.

A. M. Turing. Computing Machinery and Intelligence. *Mind, New Series*, 59(236):433–460, 1950.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Paul J. Werbos. *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis, Harvard University, Cambridge, MA, 1974.